

UNIT I

Basic Structure Of Computers: Computer Types, Functional unit, Basic operational concepts, Bus structures, Software, Performance, multiprocessors and multi computers.

Register Transfer Micro Operations: Register Transfer language. Register Transfer Bus and memory transfers, Arithmetic Micro operations, logic micro operations, shift micro operations, Arithmetic logic shift unit.

Basic Structure of Computers

Computer Architecture in general covers three aspects of computer design namely: Computer Hardware, Instruction set Architecture and Computer Organization.

Computer hardware consists of electronic circuits, displays, magnetic and optical storage media and communication facilities.

Instruction set Architecture is programmer visible machine interface such as instruction set, registers, memory organization and exception handling. Two main approaches are mainly CISC (Complex Instruction Set Computer) and RISC (Reduced Instruction Set Computer)

Computer Organization includes the high level aspects of a design, such as memory system, the bus structure and the design of the internal CPU.

Computer Types

Computer is a fast electronic calculating machine which accepts digital input, processes it according to the internally stored instructions (Programs) and produces the result on the output device. The internal operation of the computer can be as depicted in the figure below:

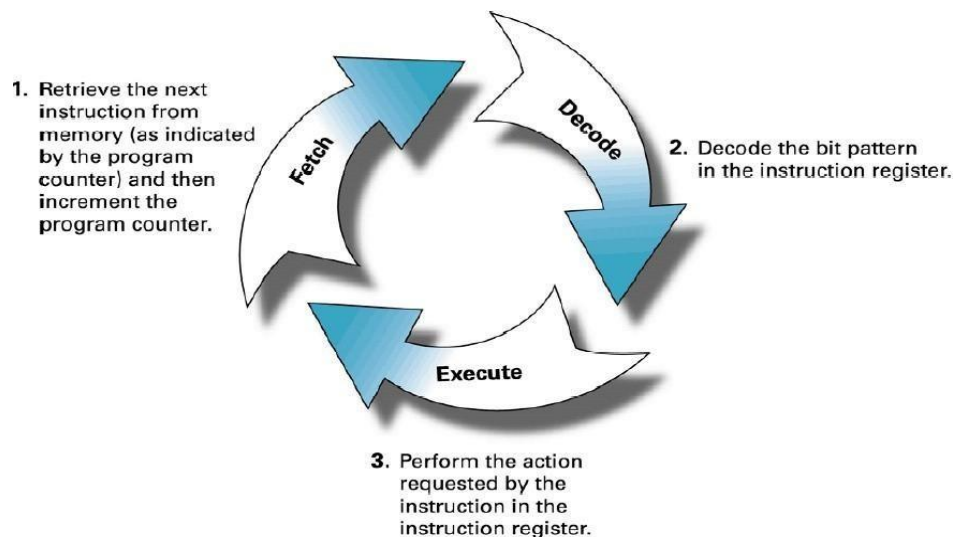


Figure 1: Fetch, Decode and Execute steps in a Computer System

The computers can be classified into various categories as given below:

- Micro Computer
- Laptop Computer
- Work Station
- Super Computer
- Main Frame
- Hand Held
- Multi core

Micro Computer: A personal computer; designed to meet the computer needs of an individual. Provides access to a wide variety of computing applications, such as word processing, photo editing, e-mail, and internet.

Laptop Computer: A portable, compact computer that can run on power supply or a battery unit. All components are integrated as one compact unit. It is generally more expensive than a comparable desktop. It is also called a Notebook.

Work Station: Powerful desktop computer designed for specialized tasks. Generally used for tasks that requires a lot of processing speed. Can also be an ordinary personal computer attached to a LAN (local area network).

Super Computer: A computer that is considered to be fastest in the world. Used to execute tasks that would take lot of time for other computers. For Ex: Modeling weather systems, genome sequence, etc

Main Frame: Large expensive computer capable of simultaneously processing data for hundreds or thousands of users. Used to store, manage, and process large amounts of data that need to be reliable, secure, and centralized.

Hand Held: It is also called a PDA (Personal Digital Assistant). A computer that fits into a pocket, runs on batteries, and is used while holding the unit in your hand. Typically used as an appointment book, address book, calculator and notepad.

Multi Core: Have Multiple Cores – parallel computing platforms. Many Cores or computing elements in a single chip. Typical Examples: Sony Play station, Core 2 Duo, i3, i7 etc.

GENERATION OF COMPUTERS

Development of technologies used to fabricate the processors, memories and I/O units of the computers has been divided into various generations as given below:

- First generation
- Second generation
- Third generation
- Fourth generation
- Beyond the fourth generation

First generation:

1946 to 1955: Computers of this generation used **Vacuum Tubes**. The computers were built using stored program concept. Ex: ENIAC, EDSAC, IBM 701.

Computers of this age typically used about ten thousand vacuum tubes. They were bulky in size, had slow operating speed, short life time and limited programming facilities.

Second generation:

1955 to 1965: Computers of this generation used the germanium **transistors** as the active switching electronic device. Ex: IBM 7000, B5000, IBM 1401. Comparatively smaller in size. About ten times faster operating speed as compared to first generation vacuum tube based computers. Consumed less power, had fairly good reliability. Availability of large memory was an added advantage.

Third generation:

1965 to 1975: The computers of this generation used the **Integrated Circuits** as the active electronic components. Ex: IBM system 360, PDP minicomputer etc. They were still smaller in size. They had powerful CPUs with the capacity of executing 1 million instructions per second (MIPS). Used to consume very less power consumption.

Fourth generation:

1976 to 1990: The computers of this generation used the **LSI chips like microprocessor** as their active electronic element. HCL horizon III, and WIPRO'S Uniplus+ HCL's Busybee PC etc.

They used high speed microprocessor as CPU. They were more user friendly and highly reliable systems. They had large storage capacity disk memories.

Beyond Fourth Generation:

1990 onwards: Specialized and dedicated **VLSI chips** are used to control specific functions of these computers. Modern Desktop PC's, Laptops or Notebook Computers.

Functional Unit

A computer in its simplest form comprises five functional units namely input unit, output unit, memory unit, arithmetic & logic unit and control unit. Figure 2 depicts the functional units of a computer system.

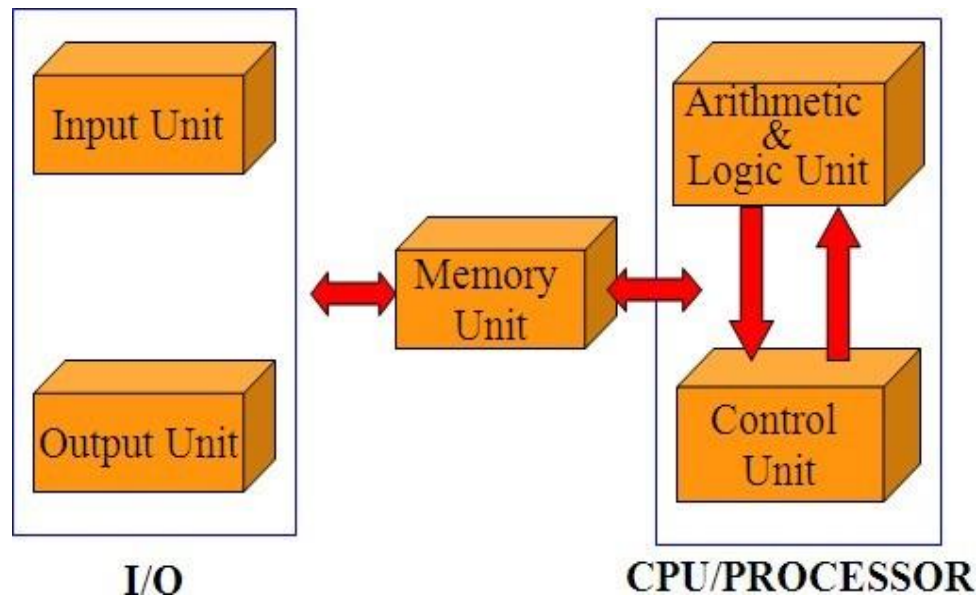


Figure 2: Basic functional units of a computer

Let us discuss about each of them in brief:

1. **Input Unit:** Computer accepts encoded information through input unit. The standard input device is a keyboard. Whenever a key is pressed, keyboard controller sends the code to CPU/Memory.

Examples include Mouse, Joystick, Tracker ball, Light pen, Digitizer, Scanner etc.

2. **Memory Unit:** Memory unit stores the program instructions (Code), data and results of computations etc. Memory unit is classified as:

- Primary /Main Memory
- Secondary /Auxiliary Memory

Primary memory is a semiconductor memory that provides access at high speed. Run time program instructions and operands are stored in the main memory. Main memory is classified again as ROM and RAM. ROM holds system programs and firmware routines such as BIOS, POST, I/O Drivers that are essential to manage the hardware of a computer. RAM is termed as Read/Write memory or user memory that holds run time program instruction and data. While primary storage is essential, it is volatile in nature and expensive. Additional requirement of memory could be supplied as auxiliary memory at cheaper cost.

Secondary memories are non volatile in nature.

3. **Arithmetic and logic unit:** ALU consist of necessary logic circuits like adder, comparator etc., to perform operations of addition, multiplication, comparison of two numbers etc.
4. **Output Unit:** Computer after computation returns the computed results, error messages, etc. via output unit. The standard output device is a video monitor, LCD/TFT monitor. Other output devices are printers, plotters etc.
5. **Control Unit:** Control unit co-ordinates activities of all units by issuing control signals. Control signals issued by control unit govern the data transfers and then appropriate operations take place. Control unit interprets or decides the operation/action to be performed.

The operations of a computer can be summarized as follows:

1. A set of instructions called a program reside in the main memory of computer.
2. The CPU fetches those instructions sequentially one-by-one from the main memory, decodes them and performs the specified operation on associated data operands in ALU.
3. Processed data and results will be displayed on an output unit.
4. All activities pertaining to processing and data movement inside the computer machine are governed by control unit.

Basic Operational Concepts

An Instruction consists of two parts, an Operation code and operand/s as shown below:

Let us see a typical instruction

OPCODE	OPERAND/s
--------	-----------

ADD LOC A, R0

This instruction is an addition operation. The following are the steps to execute the instruction:

Step 1: Fetch the instruction from main memory into the processor

Step 2: Fetch the operand at location LOC A from main memory into the processor

Step 3: Add the memory operand (i.e. fetched contents of LOC A) to the contents of register R0

Step 4: Store the result (sum) in R0.

The same instruction can be realized using two instructions as

```
Load  LOC A,R1
Add   R1, R0
```

The steps to execute the instructions can be enumerated as below:

Step 1: Fetch the instruction from main memory into the processor

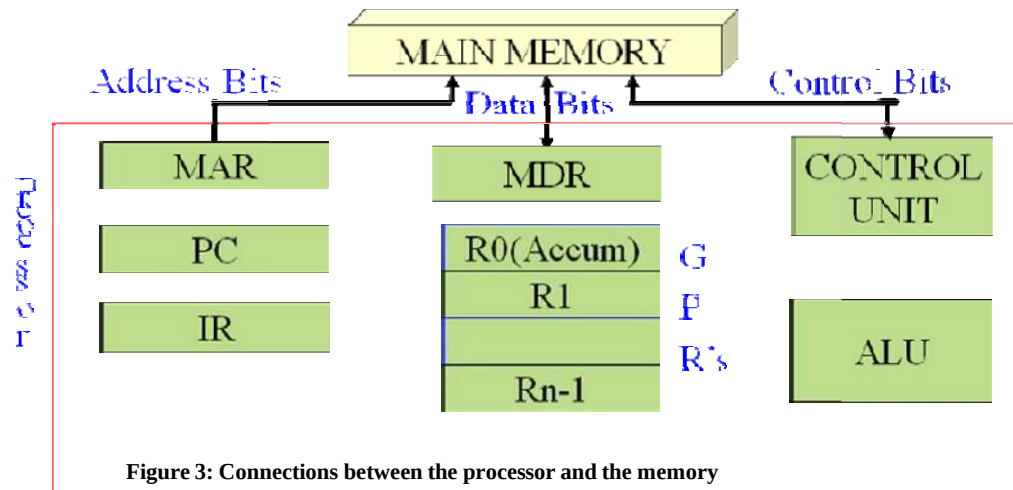
Step 2: Fetch the operand at location LOCA from main memory into the processor Register R1

Step 3: Add the content of Register R1 and the contents of register R0

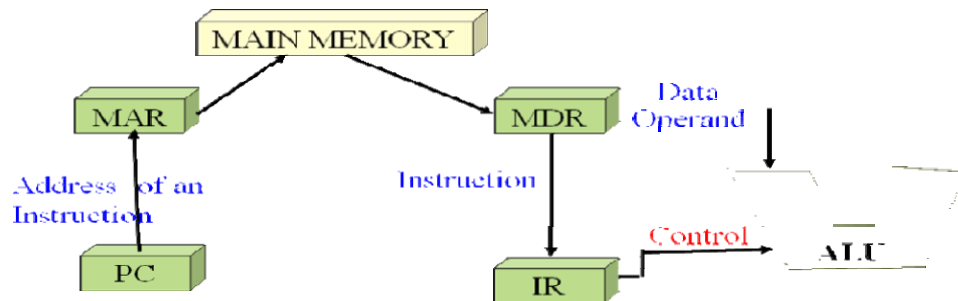
Step 4: Store the result(sum) in R0.

Figure 3 below shows how the memory and the processor are connected. As shown in the diagram, in addition to the ALU and the control circuitry, the processor contains a number of registers used for several different purposes. The instruction register holds the instruction that is currently being executed. The program counter keeps track of the execution of the program. It contains the memory address of the next instruction to be fetched and executed.

There are n general purpose registers R_0 to R_{n-1} which can be used by the programmers during writing program



The interaction between the processor and the memory and the direction of flow of information is as shown in the diagram below:



BUS STRUCTURES

Group of lines that serve as connecting path for several devices is called a bus (one bit per line). Individual parts must communicate over a communication line or path for exchanging data, address and control information as shown in the diagram below. Printer example – processor to printer. A common approach is to use the concept of buffer registers to hold the content during the transfer.

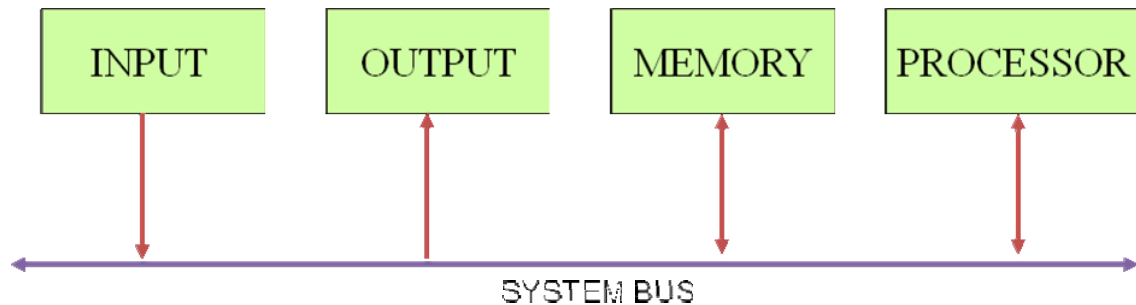


Figure 5: Single bus structure

SOFTWARE

If a user wants to enter and run an application program, he/she needs a System Software. System Software is a collection of programs that are executed as needed to perform functions such as:

- Receiving and interpreting user commands
- Entering and editing application programs and storing them as files in secondary storage devices
- Running standard application programs such as word processors, spread sheets, games etc...

Operating system - is key system software component which helps the user to exploit the below underlying hardware with the programs.

USER PROGRAM and OS ROUTINE INTERACTION

Let's assume computer with 1 processor, 1 disk and 1 printer and application program is in machine code on disk. The various tasks are performed in a coordinated fashion, which is called multitasking. $t_0, t_1 \dots t_5$ are the instances of time and the interaction during various instances as given below:

- t_0 : the OS loads the program from the disk to memory
- t_1 : program executes
- t_2 : program accesses disk
- t_3 : program executes some more
- t_4 : program accesses printer
- t_5 : program terminates

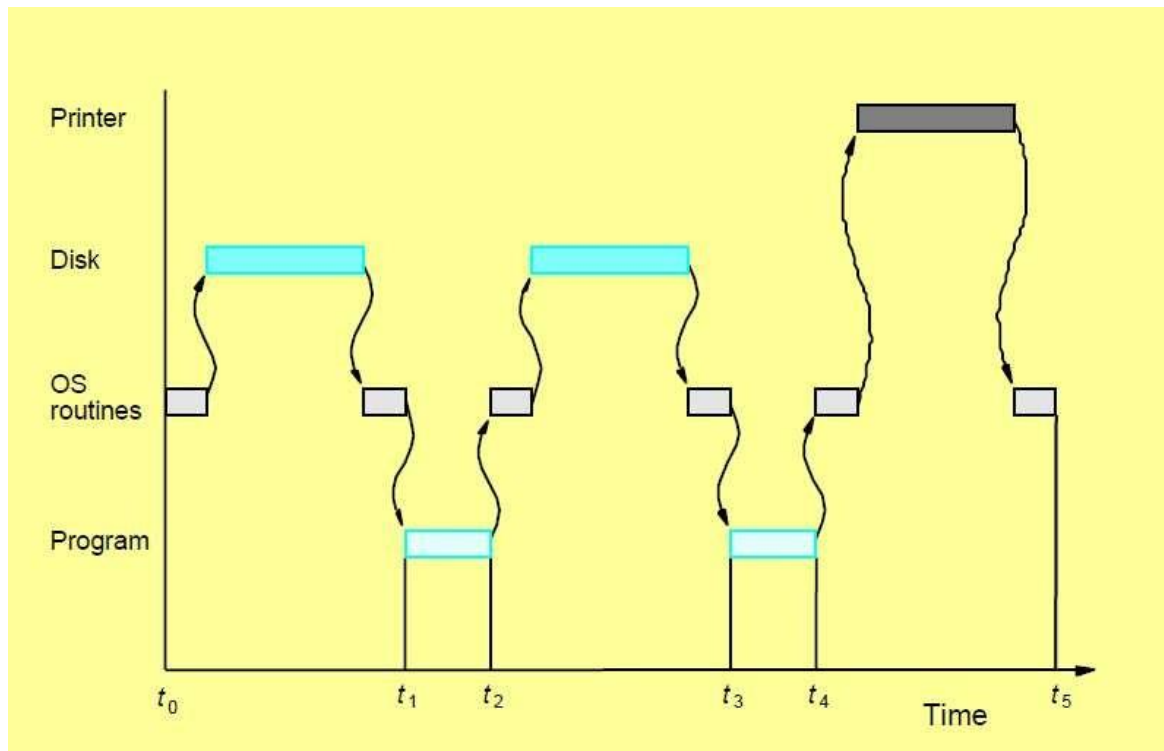


Figure 6 : User program and OS routine sharing of the processor

PERFORMANCE

The most important measure of the performance of a computer is how quickly it can execute programs. The speed with which a computer executes program is affected by the design of its hardware. For best performance, it is necessary to design the compiler, the machine instruction set, and the hardware in a coordinated way.

The total time required to execute the program is elapsed time, a measure of the performance of the entire computer system. It is affected by the speed of the processor, the disk, and the printer. The time needed to execute an instruction is called the processor time.

Just as the elapsed time for the execution of a program depends on all units in a computer system, the processor time depends on the hardware involved in the execution of individual machine instructions. This hardware comprises the processor and the memory which are usually connected by the bus.

The pertinent parts of the fig. c is repeated in fig. d which includes the cache memory as part of the processor unit.

Let us examine the flow of program instructions and data between the memory and the processor. At the start of execution, all program instructions and the required data are stored in the main memory. As the execution proceeds, instructions are fetched one by one over the bus into the processor, and a copy is placed in the cache later if the same instruction or data item is needed a second time, it is read directly from the cache.

The processor and relatively small cache memory can be fabricated on a single IC chip. The internal speed of performing the basic steps of instruction processing on chip is very high and is considerably faster than the speed at which the instruction and data can be fetched from the main memory. A program will be executed faster if the movement of instructions and data between the main memory and the processor is minimized, which is achieved by using the cache.

For example:- Suppose a number of instructions are executed repeatedly over a short period of time as happens in a program loop. If these instructions are available in the cache, they can be fetched quickly during the period of repeated use. The same applies to the data that are used repeatedly.

Processor clock:

Processor circuits are controlled by a timing signal called clock. The clock designer the regular time intervals called clock cycles. To execute a machine instruction the processor divides the action to be performed into a sequence of basic steps that each step can be completed in one clock cycle. The length P of one clock cycle is an important parameter that affects the processor performance.

Processor used in today's personal computer and work station have a clock rates that range from a few hundred million to over a billion cycles per second.

Basic performance equation:

We now focus our attention on the processor time component of the total elapsed time.

Let „ T “ be the processor time required to execute a program that has been prepared in some high-level language. The compiler generates a machine language object program that corresponds to the source program. Assume that complete execution of the program requires the execution of N machine cycle language instructions. The number N is the actual number of instruction execution and is not necessarily equal to the number of machine cycle instructions in the object program. Some instruction may be executed more than once, which in the case for instructions inside a program loop others may not be executed all, depending on the input data used.

Suppose that the average number of basic steps needed to execute one machine cycle instruction is S , where each basic step is completed in one clock cycle. If clock rate is „ R “ cycles per second, the program execution time is given by

$$T = N * S / R$$

this is often referred to as the basic performance equation.

We must emphasize that N , S & R are not independent parameters changing one may affect another. Introducing a new feature in the design of a processor will lead to improved performance only if the overall result is to reduce the value of T .

Performance measurements:

It is very important to be able to access the performance of a computer, comp designers use performance estimates to evaluate the effectiveness of new features.

The previous argument suggests that the performance of a computer is given by the execution time T, for the program of interest.

In spite of the performance equation being so simple, the evaluation of „T“ is highly complex. Moreover the parameters like the clock speed and various architectural features are not reliable indicators of the expected performance.

Hence measurement of computer performance using bench mark programs is done to make comparisons possible, standardized programs must be used.

The performance measure is the time taken by the computer to execute a given bench mark. Initially some attempts were made to create artificial programs that could be used as bench mark programs. But synthetic programs do not properly predict the performance obtained when real application programs are run. A non profit organization called SPEC- system performance evaluation corporation selects and publishes bench marks.

The program selected range from game playing, compiler, and data base applications to numerically intensive programs in astrophysics and quantum chemistry. In each case, the program is compiled under test, and the running time on a real computer is measured. The same program is also compiled and run on one computer selected as reference.

The SPEC“ rating is computed as follows.

$$\text{SPEC rating} = \frac{\text{Running time on the reference computer}}{\text{Running time on the computer under test}}$$

If the SPEC rating = 50

Multiprocessor & microprocessors:

Large computers that contain a number of processor units are called multiprocessor system. These systems either execute a number of different application tasks in parallel or execute subtasks of a single large task in parallel. All processors usually have access to all memory locations in such system & hence they are called shared memory multiprocessor systems. The high performance of these systems comes with much increased complexity and cost. In contrast to multiprocessor systems, it is also possible to use an interconnected group of complete computers to achieve high total computational power. These computers normally have access to their own memory units when the tasks they are executing need to communicate data they do so by exchanging messages over a communication network. This properly distinguishes them from shared memory multiprocessors, leading to name message-passing multi computer.

Register Transfer Language And Micro Operations:

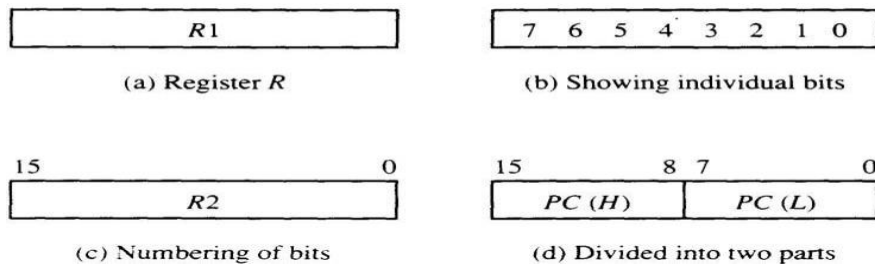
Register Transfer language:

- Digital systems are composed of modules that are constructed from digital components, such as registers, decoders, arithmetic elements, and control logic
- The modules are interconnected with common data and control paths to form a digital computer system
- The operations executed on data stored in registers are called micro operations
- A micro operation is an elementary operation performed on the information stored in one or more registers
- Examples are shift, count, clear, and load
- Some of the digital components from before are registers that implement micro operations.
- The internal hardware organization of a digital computer is best defined by specifying
 - The set of registers it contains and their functions.
 - The sequence of micro operations performed on the binary information stored
 - The control that initiates the sequence of micro operations
- Use symbols, rather than words, to specify the sequence of micro operations
- The symbolic notation used is called a register transfer language
- A programming language is a procedure for writing symbols to specify a given computational process
- Define symbols for various types of micro operations and describe associated hardware that can implement the micro operations

Register Transfer

- Designate computer registers by capital letters to denote its function
- The register that holds an address for the memory unit is called MAR
- The program counter register is called PC
- IR is the instruction register and R1 is a processor register
- The individual flip-flops in an n-bit register are numbered in sequence from 0 to n-1
- Refer to Figure 4.1 for the different representations of a register

Figure 4-1 Block diagram of register.

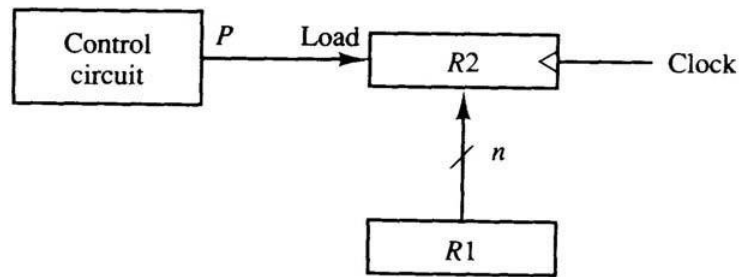


- Designate information transfer from one register to another by $R2 \leftarrow R1$
- This statement implies that the hardware is available
 - The outputs of the source must have a path to the inputs of the destination
 - The destination register has a parallel load capability
- If the transfer is to occur only under a predetermined control condition, designate it by
If ($P = 1$) then ($R2 \leftarrow R1$)
or,
 $P: R2 \leftarrow R1$,

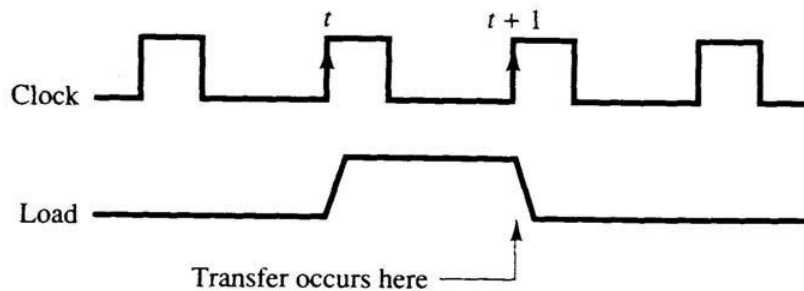
where P is a control function that can be either 0 or 1

- Every statement written in register transfer notation implies the presence of the required hardware construction

Figure 4-2 Transfer from R1 to R2 when $P = 1$.



(a) Block diagram



(b) Timing diagram

- It is assumed that all transfers occur during a clock edge transition
- All micro operations written on a single line are to be executed at the same time T : $R2 \leftarrow R1, R1 \leftarrow R2$

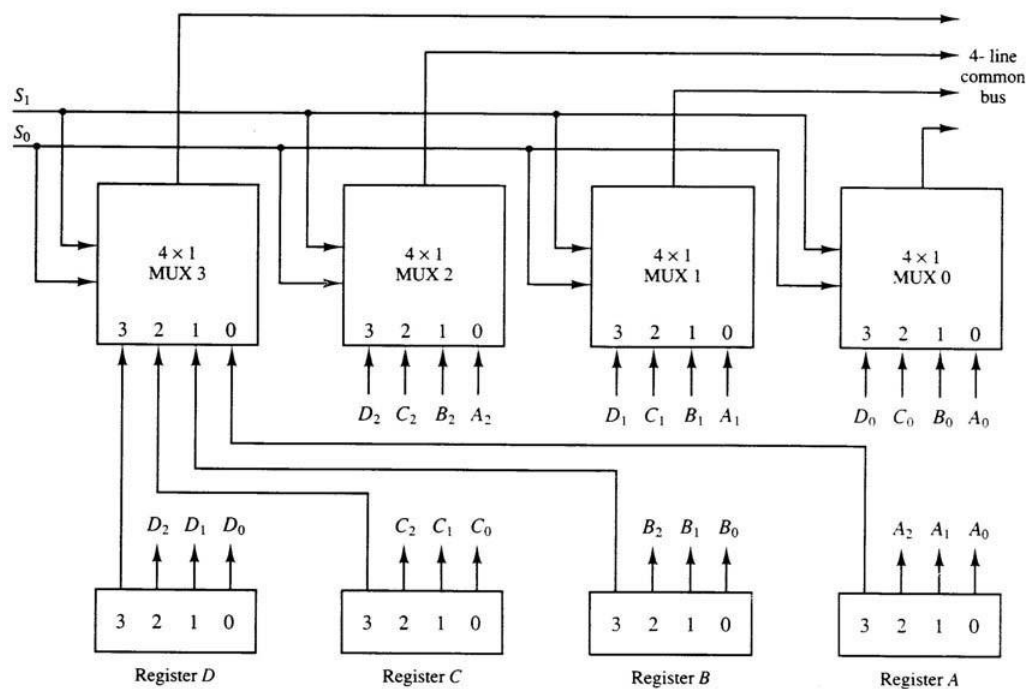
TABLE 4-1 Basic Symbols for Register Transfers

Symbol	Description	Examples
Letters (and numerals)	Denotes a register	$MAR, R2$
Parentheses ()	Denotes a part of a register	$R2(0-7), R2(L)$
Arrow $\leftarrow$	Denotes transfer of information	$R2 \leftarrow R1$
Comma ,	Separates two microoperations	$R2 \leftarrow R1, R1 \leftarrow R2$

Bus and Memory Transfers

- Rather than connecting wires between all registers, a common bus is used
- A bus structure consists of a set of common lines, one for each bit of a register
- Control signals determine which register is selected by the bus during each transfer
- Multiplexers can be used to construct a common bus
- Multiplexers select the source register whose binary information is then placed on the bus
- The select lines are connected to the selection inputs of the multiplexers and choose the bits of one register

Figure 4-3 Bus system for four registers.



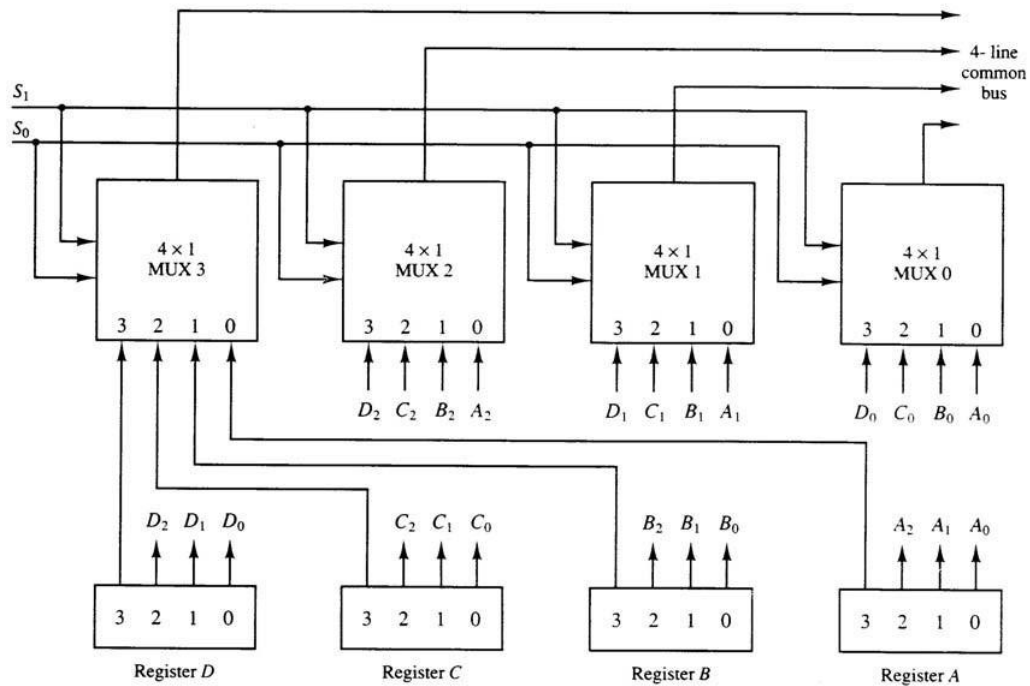
- In general, a bus system will multiplex k registers of n bits each to produce an n-line common bus
- This requires n multiplexers – one for each bit
- The size of each multiplexer must be k x 1
- The number of select lines required is log k
- To transfer information from the bus to a register, the bus lines are connected to the inputs of all destination registers and the corresponding load control line must be activated
- Rather than listing each step as
BUS $\square$ C, R1 $\square$ BUS,
use R1 $\square$ C, since the bus is implied

- To construct a common bus for four registers of n bits each using three-state buffers, we need n circuits with four buffers in each
- Only one decoder is necessary to select between the four registers
- Designate a memory word by the letter M
- It is necessary to specify the address of M when writing memory transfer operations
- Designate the address register by AR and the data register by DR
- The read operation can be stated as: Read: $DR \leftarrow M[AR]$
- The write operation can be stated as:
Write: $M[AR] \leftarrow R1$

Arithmetic Micro operations

- There are four categories of the most common micro operations:
 - **Register transfer:** transfer binary information from one register to another
 - **Arithmetic:** perform arithmetic operations on numeric data stored in registers
 - **Logic:** perform bit manipulation operations on non-numeric data stored in registers
 - **Shift:** perform shift operations on data stored in registers
- The basic arithmetic micro operations are addition, subtraction, increment, decrement, and shift
- Example of addition: $R3 \leftarrow R1 + R2$
- Subtraction is most often implemented through complementation and addition
- Example of subtraction: $R3 \leftarrow R1 + \overline{R2} + 1$ (strikethrough denotes bar on top – 1's complement of $R2$)
- Adding 1 to the 1's complement produces the 2's complement
- Adding the contents of $R1$ to the 2's complement of $R2$ is equivalent to subtracting

Figure 4-3 Bus system for four registers.



- Multiply and divide are not included as micro operations
- A micro operation is one that can be executed by one clock pulse
- Multiply (divide) is implemented by a sequence of add and shift micro operations (subtract and shift)
- To implement the add micro operation with hardware, we need the registers that hold the data and the digital component that performs the addition
- A full-adder adds two bits and a previous carry
- A binary adder is a digital circuit that generates the arithmetic sum of two binary numbers of any length
- A binary adder is constructed with full-adder circuits connected in cascade
- An n-bit binary adder requires n full-adders

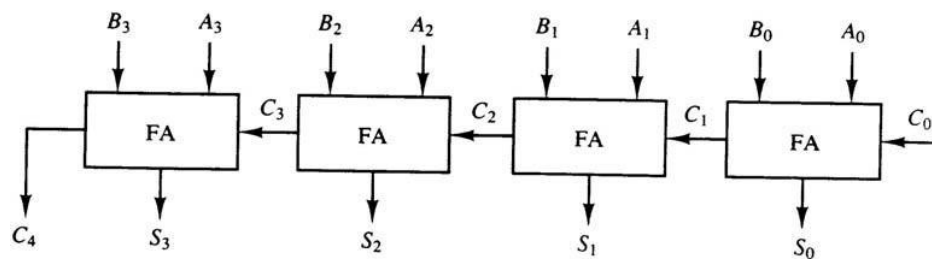


Figure 4-6 4-bit binary adder.

- The subtraction A-B can be carried out by the following steps
 - Take the 1's complement of B (invert each bit)

Computer Organization

- Get the 2's complement by adding 1
- Add the result to A
- The addition and subtraction operations can be combined into one common circuit by including an XOR gate with each full-adder

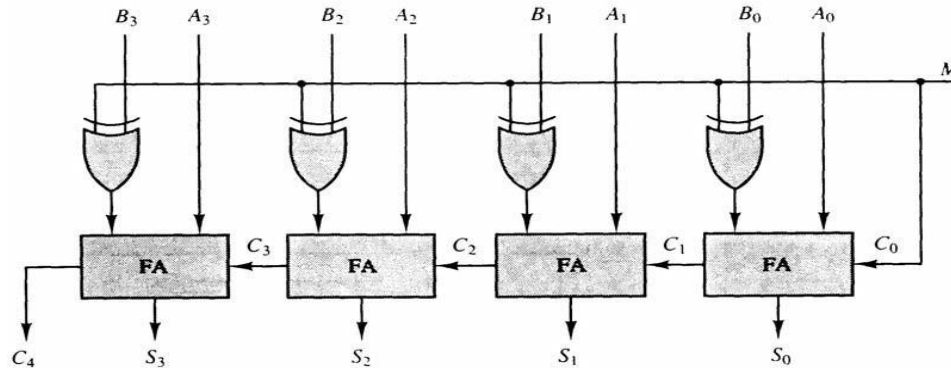


Figure 4-7 4-bit adder-subtractor.

- The increment micro operation adds one to a number in a register
- This can be implemented by using a binary counter – every time the count enable is active, the count is incremented by one
- If the increment is to be performed independent of a particular register, then use half-adders connected in cascade
- An n-bit binary incrementer requires n half-adders

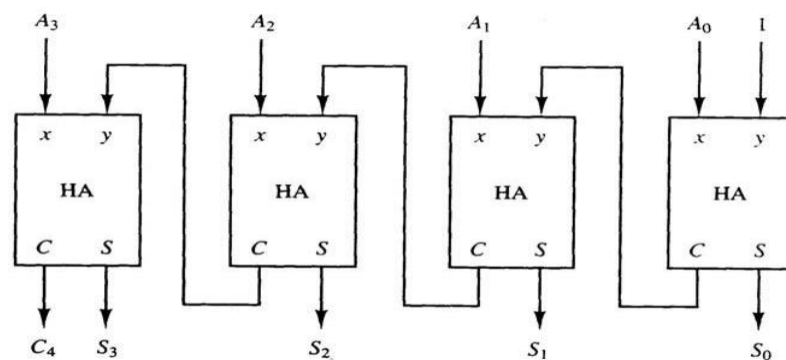


Figure 4-8 4-bit binary incrementer.

- Each of the arithmetic micro operations can be implemented in one composite arithmetic circuit
- The basic component is the parallel adder
- Multiplexers are used to choose between the different operations
- The output of the binary adder is calculated from the following sum:

$$D = A + Y + C_{in}$$

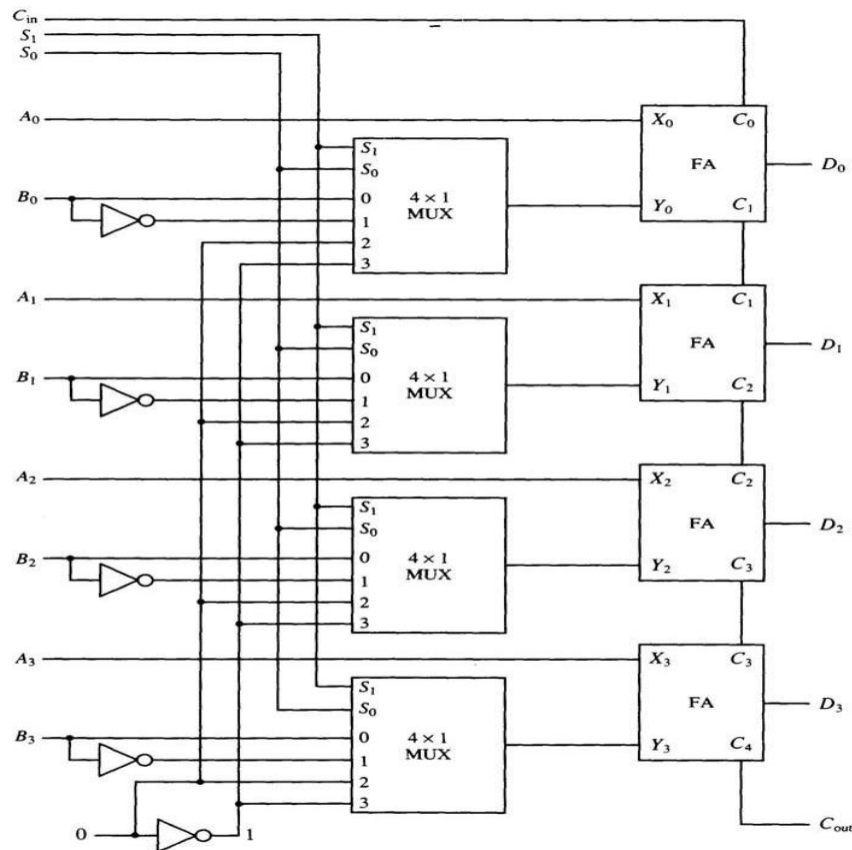


Figure 4-9 4-bit arithmetic circuit.

TABLE 4-4 Arithmetic Circuit Function Table

Select			Input Y	Output	Microoperation
S ₁	S ₀	C _{in}		D = A + Y + C _{in}	
0	0	0	B	D = A + B	Add
0	0	1	B	D = A + B + 1	Add with carry
0	1	0	$\overline{B}$	D = A + $\overline{B}$	Subtract with borrow
0	1	1	$\overline{B}$	D = A + $\overline{B}$ + 1	Subtract
1	0	0	0	D = A	Transfer A
1	0	1	0	D = A + 1	Increment A
1	1	0	1	D = A - 1	Decrement A
1	1	1	1	D = A	Transfer A

Logic Micro operations

- Logic operations specify binary operations for strings of bits stored in registers and treat each bit separately
- Example: the XOR of R1 and R2 is symbolized by

$$P: R1 \square R1 \oplus R2$$

- Example: R1 = 1010 and R2 = 1100
1010 Content of R1
1100 Content of R2
0110 Content of R1 after P = 1
- Symbols used for logical micro operations:
 - OR: $\square$
 - AND: $\square$
 - XOR: $\oplus$
- The + sign has two different meanings: logical OR and summation
- When + is in a micro operation, then summation
- When + is in a control function, then OR
- Example:
P + Q: R1 $\square$ R2 + R3, R4 $\square$ R5 $\square$ R6
- There are 16 different logic operations that can be performed with two binary variables

TABLE 4-5 Truth Tables for 16 Functions of Two Variables

x	y	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

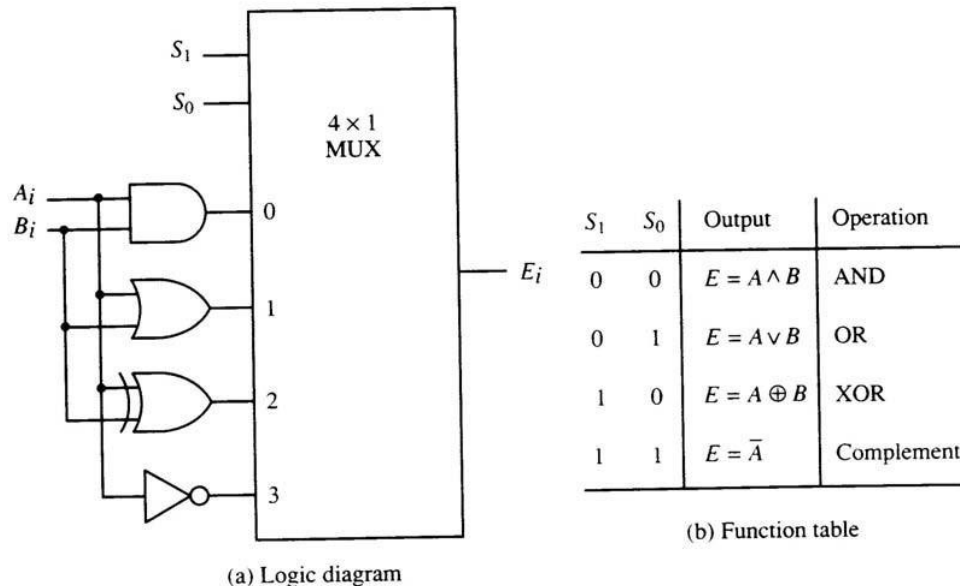
TABLE 4-6 Sixteen Logic Microoperations

Boolean function	Microoperation	Name
$F_0 = 0$	$F \leftarrow 0$	Clear
$F_1 = xy$	$F \leftarrow A \wedge B$	AND
$F_2 = xy'$	$F \leftarrow A \wedge \overline{B}$	
$F_3 = x$	$F \leftarrow A$	Transfer A
$F_4 = x'y$	$F \leftarrow \overline{A} \wedge B$	
$F_5 = y$	$F \leftarrow B$	Transfer B
$F_6 = x \oplus y$	$F \leftarrow A \oplus B$	Exclusive-OR
$F_7 = x + y$	$F \leftarrow A \vee B$	OR
$F_8 = (x + y)'$	$F \leftarrow \overline{A \vee B}$	NOR
$F_9 = (x \oplus y)'$	$F \leftarrow \overline{A \oplus B}$	Exclusive-NOR
$F_{10} = y'$	$F \leftarrow \overline{B}$	Complement B
$F_{11} = x + y'$	$F \leftarrow A \vee \overline{B}$	
$F_{12} = x'$	$F \leftarrow \overline{A}$	Complement A
$F_{13} = x' + y$	$F \leftarrow \overline{A} \vee B$	
$F_{14} = (xy)'$	$F \leftarrow \overline{A \wedge B}$	NAND
$F_{15} = 1$	$F \leftarrow \text{all 1's}$	Set to all 1's

Computer Organization

- The hardware implementation of logic micro operations requires that logic gates be inserted for each bit or pair of bits in the registers
- All 16 micro operations can be derived from using four logic gates

Figure 4-10 One stage of logic circuit.



- Logic micro operations can be used to change bit values, delete a group of bits, or insert new bit values into a register
- The selective-set operation sets to 1 the bits in A where there are corresponding 1's in B

1010	A before
<u>1100</u>	B (logic operand)
1110	A after $A \sqcup A \sqcup B$
- The selective-complement operation complements bits in A where there are corresponding 1's in B

1010	A before
<u>1100</u>	B (logic operand)
0110	A after $A \sqcup A \oplus B$
- The selective-clear operation clears to 0 the bits in A only where there are corresponding 1's in B

1010	A before
<u>1100</u>	B (logic operand)
0010	A after $A \sqcup A \sqcap B$
- The mask operation is similar to the selective-clear operation, except that the bits of A are cleared only where there are corresponding 0's in B

1010	A before
<u>1100</u>	B (logic operand)
1000	A after $A \sqcup A \sqcap B$
- The insert operation inserts a new value into a group of bits
- This is done by first masking the bits to be replaced and then Oring them with the bits to be inserted

0110 1010	A before
<u>0000 1111</u>	B (mask)
0000 1010	A after masking
0000 1010	A before
<u>1001 0000</u>	B (insert)
1001 1010	A after insertion

- The clear operation compares the bits in A and B and produces an all 0's result if the two numbers are equal

1010	A
<u>1010</u>	B
0000	$A \square A \oplus B$

Shift Micro operations

- Shift micro operations are used for serial transfer of data
- They are also used in conjunction with arithmetic, logic, and other data-processing operations
- There are three types of shifts: logical, circular, and arithmetic
- A logical shift is one that transfers 0 through the serial input
- The symbols shl and shr are for logical shift-left and shift-right by one position $R1 \square \text{shl}R1$
- The circular shift (aka rotate) circulates the bits of the register around the two ends without loss of information
- The symbols cil and cir are for circular shift left and right.

TABLE 4-7 Shift Microoperations

Symbolic designation	Description
$R \leftarrow \text{shl } R$	Shift-left register R
$R \leftarrow \text{shr } R$	Shift-right register R
$R \leftarrow \text{cil } R$	Circular shift-left register R
$R \leftarrow \text{cir } R$	Circular shift-right register R
$R \leftarrow \text{ashl } R$	Arithmetic shift-left R
$R \leftarrow \text{ashr } R$	Arithmetic shift-right R

- The arithmetic shift shifts a signed binary number to the left or right
- To the left is multiplying by 2, to the right is dividing by 2
- Arithmetic shifts must leave the sign bit unchanged
- A sign reversal occurs if the bit in R_{n-1} changes in value after the shift
- This happens if the multiplication causes an overflow
- An overflow flip-flop V_s can be used to detect

$$\text{the over flow } V_s = R_{n-1} \oplus R_{n-2}$$

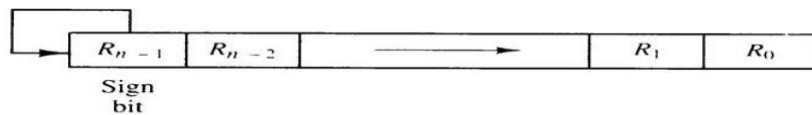


Figure 4-11 Arithmetic shift right.

- A bi-directional shift unit with parallel load could be used to implement this
- Two clock pulses are necessary with this configuration: one to load the value and another to shift
- In a processor unit with many registers it is more efficient to implement the shift operation with a combinational circuit
- The content of a register to be shifted is first placed onto a common bus and the output is connected to the combinational shifter, the shifted number is then loaded back into the register
- This can be constructed with multiplexers.

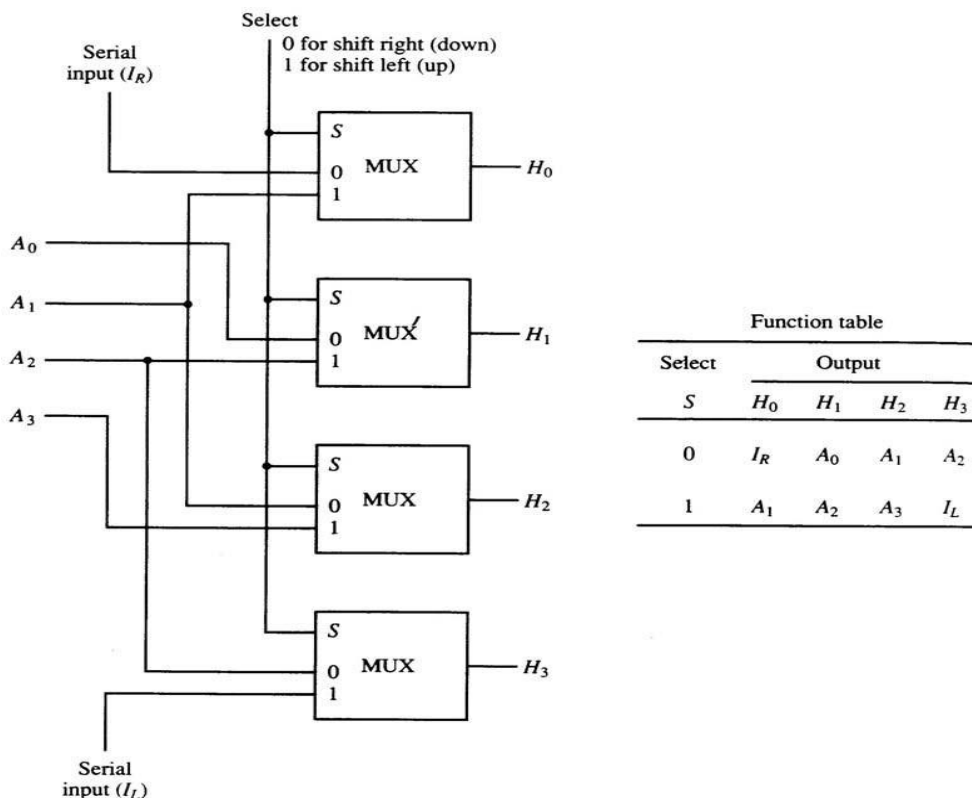


Figure 4-12 4-bit combinational circuit shifter.

Arithmetic Logic Shift Unit

- The arithmetic logic unit (ALU) is a common operational unit connected to a number of storage registers
- To perform a micro operation, the contents of specified registers are placed in the inputs of the ALU
- The ALU performs an operation and the result is then transferred to a destination register
- The ALU is a combinational circuit so that the entire register transfer operation from the source registers through the ALU and into the destination register can be performed during one clock pulse period.

Figure 4-13 One stage of arithmetic logic shift unit.

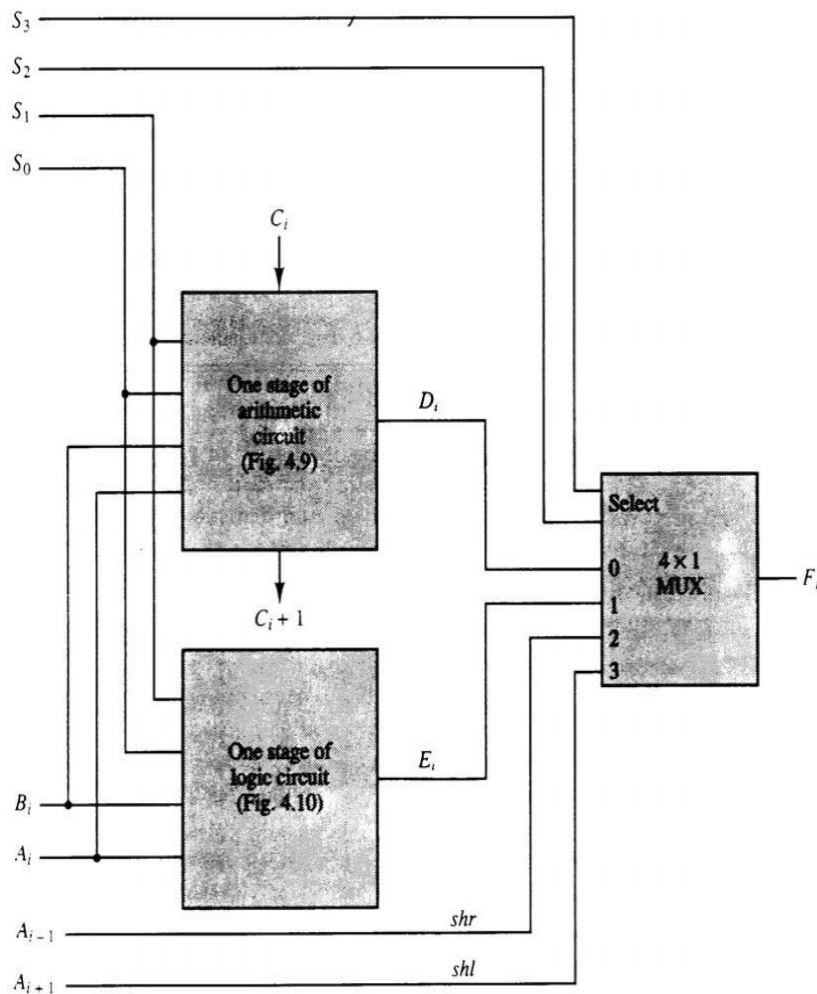


TABLE 4-8 Function Table for Arithmetic Logic Shift Unit

Operation select					Operation	Function
S_3	S_2	S_1	S_0	C_{in}		
0	0	0	0	0	$F = A$	Transfer A
0	0	0	0	1	$F = A + 1$	Increment A
0	0	0	1	0	$F = A + B$	Addition
0	0	0	1	1	$F = A + B + 1$	Add with carry
0	0	1	0	0	$F = A + \overline{B}$	Subtract with borrow
0	0	1	0	1	$F = A + \overline{B} + 1$	Subtraction
0	0	1	1	0	$F = A - 1$	Decrement A
0	0	1	1	1	$F = A$	Transfer A
0	1	0	0	$\times$	$F = A \wedge B$	AND
0	1	0	1	$\times$	$F = A \vee B$	OR
0	1	1	0	$\times$	$F = A \oplus B$	XOR
0	1	1	1	$\times$	$F = \overline{A}$	Complement A
1	0	$\times$	$\times$	$\times$	$F = \text{shr } A$	Shift right A into F
1	1	$\times$	$\times$	$\times$	$F = \text{shl } A$	Shift left A into F